

CSE331 - Assignment #2: A Divide-and-Conquer Approach to the Traveling Salesman Problem using K-means Clustering

Hojun Lee (20211277)

UNIST

South Korea

hojunlee@unist.ac.kr

Abstract

The Traveling Salesman Problem (TSP) is a classic NP-hard combinatorial optimization problem with wide-ranging applications. Exact algorithms are computationally infeasible for large instances, necessitating the use of heuristics and approximation algorithms. This paper reviews established TSP algorithms, including the exact Held-Karp algorithm, the MST-based 2-approximation algorithm, and the 1.5-approximation Christofides algorithm. We then propose a novel algorithm, the Cluster-Based TSP Approximation, which employs a divide-and-conquer strategy. This method partitions the problem space using K-means clustering, solves the resulting subproblems optimally with the Held-Karp algorithm, and merges the local tours to form a global solution. Experimental results on standard TSP benchmarks, including the 100,000-city Mona Lisa dataset, demonstrate that our proposed algorithm significantly outperforms standard Christofides implementations in runtime for large-scale instances while maintaining a high-quality approximation ratio, achieving a superior balance between computational efficiency and solution accuracy.

1 Introduction

The Traveling Salesman Problem (TSP) asks for the shortest possible route that visits each city in a given list exactly once and returns to the origin city [4]. Formally, it seeks a minimum-weight Hamiltonian cycle in a weighted, complete graph. As an NP-hard problem, the time required to find an optimal solution grows exponentially with the number of cities, rendering exact algorithms impractical for all but the smallest instances [2]. This has led to the development of various approximation algorithms that provide a trade-off between solution quality and computational time. This report examines three foundational algorithms as required by the course manual: the exact Held-Karp algorithm, the MST-based 2-approximation algorithm, and the Christofides algorithm, which guarantees a solution within 1.5 times the optimal length. While effective, these algorithms have limitations. The Held-Karp algorithm's exponential complexity restricts its use to small problem sizes, and the Christofides algorithm's performance degrades on very large datasets due to its canonical cubic-time complexity. To address this gap, we propose a novel hybrid algorithm, the Cluster-Based TSP Approximation. Our approach is motivated by the observation that optimal tours are typically composed of connections between geographically proximate nodes. By leveraging a divide-and-conquer paradigm that combines K-means clustering with the exact Held-Karp solver for local subproblems, we aim to achieve significant runtime improvements on large-scale instances while preserving high-quality solutions.

2 Problem Statement

The TSP is NP-hard, making the computation of optimal solutions computationally intractable for real-world applications involving a large number of cities. While approximation algorithms provide feasible alternatives, they too face challenges. High-quality methods like the Christofides algorithm, which provides one of the best worst-case guarantees, are bottlenecked by computationally expensive subroutines. Specifically, the minimum-weight perfect matching step requires $O(n^3)$ time, which is prohibitive for instances with tens of thousands of nodes or more [3]. This creates a need for algorithms that can scale to large datasets without sacrificing too much solution quality. Our work addresses this challenge by proposing a novel framework that leverages graph clustering to decompose a large problem into smaller, more manageable subproblems. The core objective is to design an algorithm with a sub-cubic time complexity that can produce high-quality approximate solutions for a large-scale TSP instances where existing approximation algorithms fail to terminate in a reasonable time.

3 Existing Algorithms

3.1 Held-Karp Algorithm

The Held-Karp algorithm finds the exact solution to the TSP using dynamic programming. Its time complexity analysis is as follows:

- **DP State:** The core idea is to store the cost of the shortest path from a starting city (e.g., city 1) to every other city 'i', having visited a specific subset of cities represented by a bitmask 'mask'. The DP table entry is $dp[mask][i]$.
- **Space Complexity:** There are 2^n possible subsets of cities and n possible ending cities. Therefore, the total number of states to store is $O(n \cdot 2^n)$, which defines the space complexity. This exponential space requirement makes it impractical for instances larger than 20-25 cities.
- **Time Complexity:** To compute each of the $O(n \cdot 2^n)$ states, the algorithm must iterate through all possible previous cities 'j' within the current subset 'mask'. This inner loop takes $O(n)$ time. The total time complexity is the product of the number of states and the transition time, resulting in $O(n^2 \cdot 2^n)$.

The recurrence relation is defined as:

$$dp[mask][i] = \min_{\substack{j \neq i \\ (mask \text{ contains } j)}} \{ dp[mask \setminus \{i\}][j] + d(j, i) \}$$

where $d(j, i)$ is the distance between city j and city i .

3.2 MST-based 2-Approximation Algorithm

This algorithm guarantees a solution no more than twice the optimal length for metric TSP instances. Its time complexity is derived from its constituent steps:

- (1) **Compute MST:** Construct a Minimum Spanning Tree (MST) of the graph. For a complete graph with n vertices, using Prim's algorithm with a standard binary heap takes $O(n^2)$ time.
- (2) **Create Eulerian Multigraph:** Duplicate every edge in the MST to ensure all vertices have an even degree. This involves iterating through the $n - 1$ edges of the MST and takes $O(n)$ time.
- (3) **Find Eulerian Circuit:** An Eulerian circuit in the resulting multigraph (which has $2(n - 1)$ edges) can be found using Hierholzer's algorithm in time linear to the number of edges, i.e., $O(n)$.
- (4) **Convert to Hamiltonian Circuit:** Traverse the Eulerian circuit and use shortcuts to bypass already-visited vertices. This final conversion step takes $O(n)$ time.

The overall time complexity is dominated by the most expensive step, which is the MST construction. Therefore, the algorithm runs in $O(n^2)$ time.

3.3 Christofides 1.5-Approximation Algorithm

The Christofides algorithm provides an improved 1.5-approximation guarantee for the metric TSP. Its complexity analysis is as follows:

- (1) **Compute MST:** Find an MST, T , of the graph. This step takes $O(n^2)$ time.
- (2) **Identify Odd-Degree Vertices:** Let O be the set of vertices with an odd degree in T . This can be done by traversing the MST in $O(n)$ time.
- (3) **Find Minimum-Weight Perfect Matching:** On the subgraph induced by the vertices in O , find a minimum-weight perfect matching, M . This is the algorithm's computational bottleneck. The best-known algorithms for general graphs, such as the blossom algorithm, require $O(n^3)$ time.
- (4) **Form Eulerian Multigraph:** Combine the edges of the MST and the matching ($H = T \cup M$). This union operation takes $O(n)$ time.
- (5) **Find Eulerian Circuit and Convert:** Find an Eulerian circuit in H and convert it to a Hamiltonian circuit using shortcuts, which takes $O(n)$ time.

The total complexity is dominated by the matching step, making the Christofides algorithm's overall time complexity $O(n^3)$.

4 Proposed Algorithm: Cluster-Based TSP Approximation

4.1 Motivation and Design Philosophy

The primary motivation for our proposed algorithm stems from a fundamental observation about the structure of TSP solutions: optimal or near-optimal tours are dominated by connections between geographically proximate nodes. Including distant vertices in the search for a local path segment is computationally inefficient. This suggests that a divide-and-conquer strategy, which decomposes the problem into localized subproblems, could be highly effective.

Existing high-quality approximation algorithms like Christofides do not explicitly leverage this locality, and their cubic time complexity becomes a major barrier for large-scale problem instances. Our goal is to create an algorithm that scales more gracefully while retaining high solution quality. Our design philosophy is a hybrid approach that combines the strengths of several techniques:

- (1) **Divide (Clustering):** We use K-means clustering to partition the set of all cities into a fixed number of geographically coherent clusters. K-means is ideal for this task as it minimizes intra-cluster variance, naturally grouping nearby points together.
- (2) **Conquer (Exact Local Solution):** For each cluster, we find the *optimal* local tour using the Held-Karp algorithm. Because we constrain the maximum cluster size, the exponential complexity of Held-Karp becomes a constant-time operation for each subproblem, allowing us to find high-quality local paths efficiently.
- (3) **Combine (DP-based Merging):** Once local tours are found, we must connect them into a single global tour. We treat each cluster as a single "super-node" and find the optimal visiting sequence for these super-nodes by solving another, much smaller TSP using dynamic programming. The final tour is constructed by stitching together the local tours according to this optimal cluster sequence.

This hierarchical approach allows us to tackle the combinatorial complexity at a local level where it is manageable.

4.2 K-means Clustering

K-means is an unsupervised machine learning algorithm used to partition a given dataset into a pre-determined number (k) of clusters. The main objective is to minimize the within-cluster sum of squares (i.e., variance), thereby grouping similar data points together. In the context of the TSP, the data points are the geographic coordinates of the cities. The algorithm operates iteratively as follows:

- (1) **Initialization:** k initial "centroids" are chosen from the data points.
- (2) **Assignment Step:** Each data point (city) is assigned to the cluster whose centroid is nearest (e.g., based on Euclidean distance).
- (3) **Update Step:** The centroid of each cluster is recalculated by taking the mean of all data points assigned to it.
- (4) **Convergence:** Steps 2 and 3 are repeated until the cluster assignments no longer change or a maximum number of iterations is reached.

The time complexity of K-means is generally expressed as $O(i \cdot k \cdot N)$, where i is the number of iterations for convergence, k is the number of clusters, and N is the number of cities. Each iteration is dominated by the assignment step, which compares every city to every cluster centroid.

4.3 Proposed Algorithm and Complexity

The overall time complexity of our proposed algorithm is $O(N^2)$, which is derived from the Master Theorem applied to its recurrence relation: $T(N) = k \cdot T(N/k) + O(N^2)$. The algorithm, detailed in

Algorithm 1, first checks if the problem size is small enough for an exact solution. If not, it partitions the points into 15 clusters using the K-means method described above, determines the optimal sequence for visiting these clusters, and then recursively computes and merges the paths for each cluster.

Algorithm 1 Cluster-Based TSP Approximation

Require: *points*: list of coordinates

Ensure: Approximate TSP tour

```

1: if  $|points| \leq 15$  then
2:   return held_karp(points) {Base case: Solve small instances exactly}
3: end if
4: clusters  $\leftarrow kmeans(points, k = 15)$  {Partition points}
5: cluster_order  $\leftarrow dp\_path(cluster\_centroids)$  {Find optimal cluster sequence}
6: final_path  $\leftarrow []$ 
7: for each cluster in cluster_order do
8:   Connect previous path end to current cluster start.
9:   subpath  $\leftarrow$  Recursively solve TSP for cluster.points.
10:  final_path.extend(subpath)
11: end for
12: return final_path

```

5 Experiments

We conducted experiments to compare the runtime and solution quality of the implemented algorithms.¹

5.1 Experimental Setup

All experiments were performed on a Google Cloud Platform ‘e2-highmem-16’ instance (16 vCPU, 128GB RAM). The Held-Karp algorithm is omitted from the tables as it failed on all tested datasets due to excessive memory requirements inherent in its $O(n \cdot 2^n)$ state space.

5.2 Implementation Notes

Per the assignment guidelines, all core algorithms were implemented from scratch. A key challenge was the minimum-weight perfect matching subroutine within the Christofides algorithm.

- **Christofides ($O(n^5)$, Custom Implementation):** Our from-scratch implementation fully complies with the assignment’s “no-library” policy. However, due to the complexity of implementing an efficient matching algorithm, its time complexity is $O(n^5)$, making it impractical for larger datasets.
- **Christofides ($O(n^3)$, Performance Benchmark):** To provide a meaningful performance comparison for our proposed algorithm against a standard, efficient Christofides implementation, we utilize a version based on the ‘networkx’ library purely as a benchmark. This benchmark is not part of the algorithms submitted for evaluation but serves to establish a realistic performance baseline that our custom $O(n^5)$ version could not provide on large-scale instances.

¹The complete source code for all implementations and experiments is available at: https://github.com/foxisdog/cse331_algorithm/tree/main/assignment2

5.3 Datasets

Experiments were performed on four standard benchmark datasets from TSPLIB [5] and the University of Waterloo’s TTD collection [1], as required by the course manual.

Table 1: Dataset Characteristics and Optimal Tour Values.

Dataset	a280	XQL662	kz9976	mona-lisa
Nodes	280	662	9,976	100,000
Optimal Tour	2,579	2,513	1,061,882	5,757,084

5.4 Runtime Comparison

Table 2 shows that our proposed Cluster-Based algorithm offers superior scalability. It achieves a 20.9x speedup over the standard $O(n^3)$ Christofides benchmark on ‘kz9976’. Crucially, the proposed Cluster-Based algorithm was the only method capable of processing the 100,000-city ‘mona-lisa’ dataset within the resource limits. The failure of other algorithms on this large-scale instance highlights their practical scalability limitations, underscoring the effectiveness of our divide-and-conquer approach for handling massive TSP instances, as anticipated for NP-hard problems.

Table 2: Runtime comparison in seconds. ‘OOM’ indicates Out-of-Memory error. ‘>24h’ indicates a timeout.

Algorithm	a280	XQL662	kz9976	mona-lisa
2-Approximation	0.089s	0.853s	491.49s	OOM
Christofides ($O(n^5)$)	1.660s	11.382s	>24h	OOM
Christofides ($O(n^3)$)	0.716s	4.349s	45,413s	OOM
Cluster-Based (Proposed)	2.121s	7.518s	2,167s	14,892s

5.5 Solution Quality Comparison

Table 3 compares the accuracy of each algorithm, presented as the approximation ratio (found tour cost as a percentage of the optimal solution). A lower percentage indicates a better quality solution. The Christofides benchmark generally yields the best accuracy for instances it can solve. Our proposed algorithm remains highly competitive, delivering an excellent approximation ratio of 109.4% on the massive ‘mona-lisa’ dataset.

Table 3: Approximation ratio to optimal tour (%). Lower is better.

Algorithm	a280	XQL662	kz9976	mona-lisa
2-Approximation	149.6%	146.7%	137.0%	N/A
Christofides ($O(n^5)$)	115.4%	114.7%	N/A	N/A
Christofides ($O(n^3)$)	115.4%	113.3%	111.4%	N/A
Cluster-Based (Proposed)	115.2%	127.9%	128.0%	109.4%

6 Conclusion

This paper presented the design, implementation, and evaluation of a novel Cluster-Based TSP Approximation algorithm. Motivated by the scalability limitations of existing exact and approximation algorithms on large-scale instances, our approach leverages a divide-and-conquer strategy that combines K-means clustering for problem decomposition and the Held-Karp algorithm for finding optimal local solutions. The experimental results validate our design philosophy. The proposed algorithm, with its $O(N^2)$ complexity, demonstrates a vastly superior runtime performance on large datasets compared to standard $O(N^3)$ Christofides implementations. It was the only algorithm capable of solving the 100,000-city problem due to its better scalability in terms of both time and memory. This demonstrates that for large-scale TSP, our Cluster-Based algorithm

provides a more practical and effective balance between computational efficiency and solution accuracy than established methods.

References

- [1] David L. Applegate, Robert E. Bixby, Vášek Chvátal, and William J. Cook. 2001. The TSP Test Data. <https://www.math.uwaterloo.ca/tsp/data/index.html>. Accessed: 2025-06-09.
- [2] Michael T. Goodrich and Roberto Tamassia. 2014. *Algorithm Design and Applications*. Wiley Publishing.
- [3] David S. Johnson and Lyle A. McGeoch. 2002. Experimental Analysis of Heuristics for the STSP. In *The Traveling Salesman Problem and Its Variations*. Springer, 369–443.
- [4] R. Matai, S. P. Singh, and M. L. Mittal. 2010. Traveling Salesman Problem: An Overview of Applications, Formulations, and Solution Approaches. *Traveling Salesman Problem, Theory and Applications* 1, 1 (2010), 1–25.
- [5] Gerhard Reinelt. 1991. TSPLIB – A Traveling Salesman Problem Library. <http://comopt.ifl.uni-heidelberg.de/software/TSPLIB95/>. Accessed: 2025-06-09.